GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-26

今日榜单

1. **Nutlope/hallmark** — CSS
   Anti-AI-slop design skill for Claude Code, Cursor, and Codex. — 3天
2. **TauricResearch/TradingAgents** — Python
   TradingAgents: Multi-Agents LLM Financial Trading Framework — 2天
3. **JetBrains/thinkrail** — TypeScript
   Vibe code with pi in a lightweight, real IDE that customises itself around the w... — 2天
4. **makecindy/cindy** — TypeScript
   Consider it done. The open-source AI agent that works out of the box ・ 想到，就能做到… — 3天
5. **affaan-m/ECC** — JavaScript
   The agent harness performance optimization system. Skills, instincts, memory, se... — NEW
6. **pbakaus/impeccable** — Rust
   The design language that makes your AI harness better at design. — 3天
7. **NVIDIA/SkillSpector** — Python
   Security scanner for AI agent skills. Detect vulnerabilities, malicious patterns... — NEW
8. **pollen-robotics/microduck** — Rust
   A Tiny biped duck robot 🦆 — 3天
9. **nightly-labs/openbot** — TypeScript
   A local-first desktop workspace for persistent AI teammates. Run Codex, Claude, ... — NEW
10. **Panniantong/Agent-Reach** — Python
    Give your AI agent eyes to see the entire internet. Read & search Twitter, Reddi... — NEW

#1 CSS 连续 3 天

hallmark

Anti-AI-slop design skill for Claude Code, Cursor, and Codex.

Stars **35,576** Forks **1,855** Today **+2,235**

https://github.com/Nutlope/hallmark

项目简介：Hallmark 是一个面向 Claude Code、Codex、Cursor、OpenCode 等 AI 编程工具的“设计技能”插件，目标是让 AI 生成的网页不再带有那种一眼可辨的“AI 味”——比如每个标题上方的小标签、被追踪大写字母的标签、标题里单独染色的词、满屏空框或纯照片、首屏撑不满、页尾千篇一律的订阅表单等。它由 Together AI 出品，本质上是一套约束 AI 设计行为的规则与素材库。

核心功能：一是从 48 套“主题材质”中抽取三套来构建页面，每套主题只定义材质（纸张、字体、色彩、表面、组件、纹理），不规定内容结构；二是按“用户来干什么”来布局页面，预订页就完成预订、商店就完成售卖、订阅页就完成订阅，动作放在该放的位置并作为页面收尾；三是内置真实浏览器 lint，专门检测那些“机器味”特征，并由一个全新上下文的审查者做最终把关；四是支持指向真实 URL、dev server 或应用自动截图，并使用 755 张开放授权实拍照片按主题挑选并署名；五是提供 `/hallmark redesign` 重建页面（保留内容与接线）和 `/hallmark audit` 只做报告两种模式。

适用场景：主要面向用 AI 编程工具快速生成前端页面的开发者、独立产品人和设计工程师，尤其是那些厌倦了“一看就是 AI 做的落地页”的人。典型场景包括快速搭建产品官网、营销页、预订/电商/订阅类页面，以及对已有页面做重新设计或设计审计。

技术亮点：最值得关注的是它的“分层门禁”设计——Floor 层是绝不发布的硬底线（对比度、焦点、减少动效、编造事实、手机布局崩坏），Reflex 层是模型容易陷入的默认套路（可在记录理由后破例），Finish 层是永不阻塞发布的工艺细节，这种分级让约束既有刚性又不死板。另外，主题作为“材质而非模板”的设计很巧妙，一个主题文件不超过 8KB、构建时只读一个，保证了同主题下不同提示词能产出不同页面，避免了模板化。

上手建议：先从 usehallmark.com 浏览 48 套主题、挑一个符合产品气质的材质，然后在你的 AI 编程工具中安装该 skill，用 `/hallmark` 命令配合具体业务提示词试跑一个页面，观察 lint 报告和审查者反馈。想贡献的话，可以从 `skills/hallmark/references/themes/` 下的主题文件入手（每套一个 ≤8KB 的单文件加配图），或补充 lint 规则与实拍照片库，README 中关于 Floor/Reflex/Finish 的分层说明是理解整个设计哲学的关键入口。

#2 Python 连续 2 天

TradingAgents

TradingAgents: Multi-Agents LLM Financial Trading Framework

Stars **115,610** Forks **22,218** Today **+1,376**

https://github.com/TauricResearch/TradingAgents

项目简介：TradingAgents 是 Tauric Research 推出的多智能体 LLM 金融交易框架，把“分析师团队”拆成多个各司其职的 LLM Agent（基本面、情绪、新闻、技术面等），模拟真实投研流程来生成交易决策。它解决的是单一 LLM 直接“拍脑袋选股”缺乏分工、缺乏可追溯依据的问题，背后有 arXiv 论文（2412.20138）支撑。

核心功能：一是多角色 Agent 协作——分析师分头研究、研究经理辩论汇总、交易员与组合经理最终决策；二是回测与 point-in-time 数据完整性，确保回测只看到当时已发布的数据，避免前视偏差；三是决策日志与记忆机制，让 Agent 参考历史决策持续迭代；四是多模型/多供应商支持（OpenAI、Claude、Qwen、DeepSeek、Ollama 等），并支持按层级分配不同模型；五是 CLI 与 HTML 报告输出，可一键跑单票或票池网格回测。

适用场景：适合量化研究者、AI+金融方向的开发者做多智能体协作实验，也适合个人投资者做辅助投研和策略回测验证。教学场景下，它也是理解“LLM Agent 编排 + 金融数据管道”的完整范例。

技术亮点：用 LangGraph 做 Agent 图编排并支持 checkpoint 断点续跑，工程化程度较高；对 look-ahead bias 的持续修复（FRED 宏观、社交情绪、SEC EDGAR 按申报时点取数）体现了对回测严谨性的重视；结构化输出约束各 Agent 的决策格式，让多 Agent 的“辩论”结果可解析、可复现。

上手建议：先读 README 的安装与 CLI 示例，配置好一个模型供应商的 API Key，用 `--ticker` 和 `--date` 跑一次单票分析看完整流程；再读 CHANGELOG 了解近期架构变动（v0.5 起有导入路径调整）。想贡献的话，可从数据供应商适配、look-ahead 校验和测试用例入手，这些是项目当前迭代最活跃的方向。注意：项目迭代非常快，文档与代码可能不同步，建议锁定 release 版本使用。

#3 TypeScript 连续 2 天

thinkrail

Vibe code with pi in a lightweight, real IDE that customises itself around the way you work — The Vibe You Need

Stars **2,512** Forks **137** Today **+1,156**

https://github.com/JetBrains/thinkrail

项目简介

ThinkRail 是 JetBrains 孵化的一款「智能体 IDE」（agentic IDE），它把 pi 编码智能体包裹进一个轻量但完整的 IDE 界面中，目标是让 AI 编程从「聊天窗口」升级为「可管理、可并行的工程工作流」。它要解决的核心痛点是：现有 AI 编程工具缺乏隔离、缺乏持久上下文、缺乏与真实工程流程（分支、PR、CI、评审）的衔接。

核心功能

- **隔离工作区**：以 git worktree 为单位切出独立工作区和分支，多个智能体可并行工作而互不干扰，完成后按需合并或删除，并自动「沉降」到项目货架区。
- **围绕智能体的真实 IDE**：Monaco 编辑器、文件树、终端、diff 视图一应俱全，支持在文件或 diff 上留下锚定评论并作为结构化上下文发给聊天。
- **流式多会话聊天**：每个工作区可开多个 pi 会话，各自独立模型与 token/成本显示，支持中途引导、排队追问、子智能体委派和共享计划。
- **Specs 作为事实来源**：以 SPEC.md 文件构成连接图，智能体通过专用 spec_* 工具读写维护，让「意图」跨会话留存。
- **技能与工作流**：可复用的 Agent 技能，以及后续规划中的完整扩展机制，让 IDE 能按你的工作方式自我定制。

适用场景

适合需要并行推进多个 AI 编码任务的开发者——比如同时让智能体处理多个功能分支、修复任务或实验性改动，而不污染主工作区。也适合重视「上下文延续」的团队：Specs 让项目意图沉淀下来，PR/CI 集成让 AI 产出直接进入真实评审流程。对想尝鲜 JetBrains 下一代 AI IDE 形态的工程师尤其有吸引力。

技术亮点

- **基于 git worktree 的隔离模型**是设计核心，把「并行智能体」落到版本控制原语上，天然支持分支管理和干净合并。
- **Specs 图谱 + 专用工具**：用结构化文档图而非一次性对话来承载项目知识，是解决 LLM 上下文遗忘的一种工程化思路。
- **深度集成 GitHub PR 生命周期**（CI 检查、评审决策、合并方式记忆、失败转交智能体），把 AI 输出接入真实交付链路。
- 项目处于 JetBrains incubator 阶段，TypeScript 实现，围绕开源 `pi` 智能体构建富界面。

上手建议

先访问官网 thinkrail.ai 和博客了解产品定位与安装方式，从「打开一个 git 仓库 → 切出 worktree 工作区 → 发起一个 `pi` 会话」的最小流程开始体验。想贡献代码的话，建议先读 README 剩余部分（被截断的技能/扩展章节）和仓库文档，从 issue 列表里找 incubator 阶段常见的界面或扩展支持类任务切入；由于项目较新（今日新增 1156 星），关注其迭代节奏和路线图（扩展支持）比急于提交大改动更稳妥。

#4 TypeScript 连续 3 天

cindy

Consider it done. The open-source AI agent that works out of the box · 想到，就能做到。开源、开箱即用的 AI Agent。

Stars **6,048** Forks **757** Today **+1,268**

https://github.com/makecindy/cindy

项目简介：Cindy 是一个开箱即用的开源 AI Agent 桌面/移动客户端，主打"想到就能做到"——让 AI 直接在你自己的电脑上操作真实文件、浏览器、已登录应用来完成实际工作。它解决的是通用 AI 助手"只能聊天、不能干活"的痛点，把多个 Agent 框架（Claude Code、Codex）和模型统一到一个能真正执行任务的入口里。

核心功能： - **多 Harness × 多模型自由混搭**：可在一个任务中切换 Claude Code、Codex 等框架与不同模型，工作区、记忆、技能保持连续，甚至能让不同 agent 并行规划、执行、评审。 - **本地真实执行**：在自己机器上操作真实文件、浏览器、电脑和手机，并能从 IM 和定时任务触发工作。 - **可塑造的扩展体系**：记忆（纠正一次即长期生效）、技能（教一次到处复用）、自动化定时任务、MCP 接入内部系统、插件改造 UI 与交互。 - **灵活的模型接入**：支持官方云服务、复用已付费的 Claude Code / Codex Coding Plan（不重复计费）、自带 API Key 或本地模型。

适用场景：面向希望把 AI 深度嵌入日常工作流的开发者、知识工作者和团队——比如让 agent 自动处理项目代码、整理文件、跨应用执行重复任务、按日程自动跑工作流并回报结果。也适合想审计、fork、二次开发自己 AI 工具链的技术团队。

技术亮点：采用 pnpm monorepo 组织，桌面端基于 Electron、移动端基于 Expo/React Native，共享包抽离了认证、设备互联、Agent 编排、模型提供方等能力；工具二进制（claude-code、codex、ripgrep）按平台在安装时下载，Android 平台工具还做了版本锁定与 sha256 校验。值得注意的是**后端服务不在本仓库**，开源的是客户端，这种"客户端开源 + 云服务闭源"的模式兼顾了可审计性与商业可持续性。

上手建议：普通用户直接从 cindy.app 下载安装即可，登录可选"跳过登录"使用本地 agent（但失去服务端能力）；贡献者需准备 Node 22.x、pnpm 10.x 和 Git LFS，克隆后执行 `git lfs pull && pnpm install`，再参考 `CONTRIBUTING.en.md` 和开发入口命令（如 `pnpm`

restart:desktop:remote --region=global）启动。建议先跑通本地 agent 模式理解架构，再深入 packages/* 的编排与模型层。

#5 JavaScript NEW

ECC

The agent harness performance optimization system. Skills, instincts, memory, security, and research-first development for Claude Code, Codex, Opencode, Cursor and beyond.

Stars **289,480** Forks **42,885** Today **+853**

https://github.com/affaan-m/ECC

项目简介：ECC 是一个面向 AI 编程代理（agent）的"性能优化操作系统"，为 Claude Code、Codex、Cursor 等主流 AI 编程工具提供统一的增强层。它试图解决的核心问题是：各家 AI 编程代理能力参差不齐、缺乏记忆与安全边界，ECC 通过插件化方式统一补齐这些短板。

核心功能：一是 Skills/Instincts 机制，为代理注入可复用的技能与"直觉"行为模式；二是持久化 Memory 层，让代理跨会话保留上下文；三是安全防护（AgentShield 包），约束代理行为、防止越权操作；四是"研究优先"的开发范式，引导代理先调研再动手；五是以插件形式（ecc@ecc）一键接入多种宿主工具。

适用场景：重度使用 Claude Code、Cursor 等 AI 编程代理的开发者，尤其是希望代理更"懂项目"、少犯重复错误、且对代理自动执行有安全顾虑的团队。也适合想在不同代理工具间保持一致工作流的用户。

技术亮点：以 JavaScript 为主、多语言生态（Shell/TS/Python/Go 等徽章暗示跨语言集成）的插件架构，通过 npm 包（`ecc-universal`、`ecc-agentshield`）与 GitHub App 双渠道分发，README 提供 16 种语言版本，国际化与分发工程做得相当成熟。

上手建议：从官方 `ecc.tools` 的 guided setup 或 Claude Code 原生插件命令入手，只走 README 明示的官方渠道（仓库、npm、GitHub App），警惕第三方镜像。想贡献的话，先读多语言 README 中的英文主文档理解插件架构，再从 Skills 或文档翻译等低门槛方向切入。

（注：28 万星标与单日 853 增量对常规开源项目而言异常偏高，建议实际使用前自行核验项目活跃度与代码质量。）

#6 Rust 连续 3 天

impeccable

The design language that makes your AI harness better at design.

Stars **88,751** Forks **5,293** Today **+1,551**

https://github.com/pbakaus/impeccable

项目简介：Impeccable 是一套面向 AI 编程助手的设计语言/技能包，旨在解决 AI 生成前端界面时千篇一律、缺乏设计感的问题。它从 Anthropic 官方的 frontend-design skill 出发，通过结构化命令和检测规则，让 Claude 等 agent 产出更有品味的界面。

核心功能：一是 `/impeccable init` 一次性梳理产品事实并写入 `PRODUCT.md`，为后续命令提供上下文；二是 24 个设计命令（polish、audit、critique、bolder、quieter 等），构成与 AI 共享的设计词汇表；三是 62 条确定性检测规则，无需 LLM 和 API key 即可在 CLI 和浏览器扩展中运行；四是 live/generate 模式支持在浏览器里实时迭代视觉变体。

适用场景：主要面向用 AI 生成前端代码的开发者、设计师和独立创业者，尤其适合从零搭建产品界面、对已有页面做设计审查与打磨、以及需要统一设计系统时使用。也适合团队作为 AI 编码流程中的设计质量守门环节。

技术亮点：核心引擎用 Rust 写成自包含二进制，首次运行自动下载到 `~/.impeccable/bin/`，无需额外运行时；把"确定性规则检测"和"LLM 主观评审"分层，前者可离线、可复现，是工程化设计质量的有趣尝试；同时明确列出 anti-patterns（如滥用 Inter、紫蓝渐变、卡片套卡片），把"AI 味"显式化。

上手建议：在项目根目录运行 `npx impeccable install`，再在 AI 工具中执行 `/impeccable init` 即可开始；建议先读 impeccable.style 文档和 Neo Mirai 案例了解前后对比效果。想贡献可从 62 条检测规则入手——补充新规则门槛低、收益直接，也便于理解整个引擎的判定逻辑。

#7 Python NEW

SkillSpector

Security scanner for AI agent skills. Detect vulnerabilities, malicious patterns, security risks, prompt injection, data exfiltration, and supply-chain risks in Claude Code, Codex, and MCP skills before you install them.

Stars **21,891** Forks **1,918** Today **+203**

https://github.com/NVIDIA/SkillSpector

项目简介：SkillSpector 是 NVIDIA 开源的 AI Agent 技能安全扫描器，专门用于在安装 Claude Code、Codex、MCP 等 Agent 技能前检测安全风险。它解决的是当前 Agent 技能生态"默认信任、缺乏审核"的问题——根据项目对 31,132 个技能的分析，26.1% 存在漏洞、5.2% 疑似恶意，这个比例相当惊人。

核心功能：一是覆盖 17 大类、71 种漏洞模式的检测，包括提示注入、数据外泄、提权、供应链攻击、MCP 工具投毒等；二是两阶段分析，先做快速静态扫描，再可选地调用 LLM 做语义评估；三是支持 Git 仓库、URL、zip、目录、单文件等多种输入，并输出终端/JSON/Markdown/SARIF 多种格式；四是实时查询 OSV.dev 的 CVE 数据（带离线回退），以及基于指纹或 glob 规则的误报抑制基线。

适用场景：主要面向在 Claude Code、Codex CLI、Gemini CLI 等环境中安装第三方技能的开发者，以及需要对企业内部 Agent 技能做安全准入审核的团队。典型用法是在安装技能前跑一次扫描，或把它嵌入 CI/发布流水线作为门禁——NVIDIA 自己的 Verified Skills 管道就是这么用的。

技术亮点：检测深度上不只做正则匹配，还用到了 AST 分析、污点追踪（taint tracking）和 YARA 签名，能识别动态数据流层面的风险；工程上强调"fail-closed"的资源边界控制（对 bundle、解析器、嵌套产物设上限），避免扫描器本身被恶意输入拖垮；另外提供了 Pi、OpenCode 扩展，可以让 Agent 在会话内自扫描技能，形成"用 Agent 保护 Agent"的闭环。

上手建议：最快的方式是用 `uv tool install` 从 Git 直接安装 CLI，然后对一个本地技能目录跑 `skillspector scan ./my-skill/ --no-llm` 先看纯静态结果；需要语义分析再配置 provider 和 API key。想深入或贡献代码，建议先读 `docs/DEVELOPMENT.md` 了解分析器管道的架构和扩展方式，从新增检测模式或误报抑制规则入手是比较友好的切入点。

#8 Rust 连续 3 天

microduck

A Tiny biped duck robot 🦆

Stars **12,242** Forks **1,533** Today **+844**

https://github.com/pollen-robotics/microduck

项目简介：Microduck 是 Pollen Robotics 开源的微型双足机器人（约 25cm、800g）的"大脑"软件仓库，负责让这台小鸭子机器人通过强化学习策略实现行走、翻滚、捡拾、摔倒后自主站起等动作。它解决的是把 sim2real 训练出的神经网络策略真正部署到资源受限嵌入式硬件上的完整工程问题。

核心功能：一是 50Hz 控制循环驱动 15 个舵机执行神经网络策略（ONNX 加载）；二是多模式切换，如装上轮子后加载"滚轮大脑"；三是完整的 OTA 更新机制，支持签名验证、健康门控与回滚，避免变砖；四是蓝牙手柄操控、WebRTC 摄像头推流、深度传感器服务等外设集成；五是可通过蓝牙 `duckctl` 免网络远程控制。

适用场景：一是购买了 Microduck 硬件的用户日常使用与配置；二是机器人/强化学习研究者基于真机做策略部署与 sim2real 验证；三是嵌入式 Rust 开发者学习多守护进程架构、OTA 与实时控制环设计；四是教育场景下作为具身智能入门平台。

技术亮点：整个系统用纯 Rust 无框架、单 workspace 构建，拆分为 `robotd`（控制环与电机总线）、`updaterd`（签名更新与回滚）、`configd`、`btd`、`padd`、`mediad`、`tofd` 等守护进程，统一通过 Unix socket 上的 JSON-RPC 契约通信，客户端（App、控制台、手柄、脚本）调用完全一致；策略训练在隔壁 `microduck_rl` 仓库完成（MuJoCo + PPO + 域随机化 + ONNX 导出），训练与部署清晰解耦；还提供 `duck-sim` 让真实守护进程跑在 MuJoCo 仿真身体上，无硬件也能开发。

上手建议：硬件用户从 `docs/robot/cheatsheet.md` 的 `robotctl` 命令开始；开发者先读 `docs/design/architecture.md` 一页看懂守护进程与总线，再按 `install-dev.md` 把空板刷成可接收分支构建的机器人；想改策略则转到 `microduck_rl` 仓库，无硬件时用 `scripts/duck-sim` 在仿真中调试，贡献流程参见 `CONTRIBUTING.md`。

#9 TypeScript NEW

openbot

A local-first desktop workspace for persistent AI teammates. Run Codex, Claude, and Grok with dedicated workspaces, task queues, file sharing, browser control, and agent-to-agent collaboration.

Stars **1,249** Forks **168** Today **+312**

https://github.com/nightly-labs/openbot

项目简介：OpenBot 是一个本地优先（local-first）的桌面工作区，让多个 AI 编程智能体（Codex、Claude、Grok 等）作为"持久化队友"长期驻留，各自拥有独立工作区和会话。它解决的核心问题是：目前各家 AI 编码工具彼此割裂、会话难以持久、无法协作，OpenBot 把它们统一到一个桌面应用里协同工作。

核心功能：① 多智能体并行运行，每个 agent 有独立工作区、模型、推理配置和上下文监控（超长自动压缩）；② FIFO 任务队列支持暂停/恢复/取消且崩溃安全；③ agent 之间的消息、回复、文件传输和共享频道协作（含任务委派、重新分配）；④ 内嵌持久浏览器，agent 可打开、检查并操控网页；⑤ 通过 ACP 协议接入 OpenCode、Gemini、Cursor、Cline 等，并支持 Ollama/LM Studio 本地模型。

适用场景：适合需要让多个 AI 智能体分工完成复杂开发任务的开发者——比如一个 agent 写代码、一个跑测试、一个查文档并互相传文件；也适合想用本地模型、注重数据隐私但又需要联网能力的团队。移动端创建 agent 的说明暗示还有远程操控场景。

技术亮点：基于 TypeScript 的 Electron 类桌面架构，用 ACP（Agent Client Protocol）作为统一接入层兼容多种 CLI agent；通过子进程管理 `cua-driver` 实现跨平台 Computer Use；账号后端用 Cloudflare Workers + D1，体现"本地优先但非纯离线"的混合设计。值得注意的是它默认以 `danger-full-access` + 永不审批策略运行 agent，安全边界完全依赖用户信任，这是设计上的激进取舍。

上手建议：从 GitHub Releases 下载对应平台安装包即可（macOS/Windows/Linux），Linux 用户需注意 AppArmor 配置。首次启动会要求明确授权，建议先在隔离环境或备份前提下试用。想贡献的话，先读 Security 章节理解权限模型，再关注 CI 与 ACP 接入层的扩展点——这是项目最核心也最值得参与的部分。

#10 Python NEW

Agent-Reach

Give your AI agent eyes to see the entire internet. Read & search Twitter, Reddit, YouTube, GitHub, Bilibili, XiaoHongShu — one CLI, zero API fees.

Stars **103,165** Forks **9,147** Today **+723**

https://github.com/Panniantong/Agent-Reach

项目简介：Agent Reach 是一个 Python 编写的 CLI 工具，为 Claude Code、Codex 等 AI Agent 补齐“够不着”的互联网能力——登录墙后的平台、视频内容、中文平台和结构化数据。它把各平台背后的抓取工具预先装好测好，让 Agent 一条命令即可访问 Twitter、Reddit、YouTube、GitHub、B站、小红书等，且无需支付 API 费用。

核心功能：一是统一的多平台渠道入口（`agent-reach channels` 列出可用渠道），覆盖 X、Reddit、YouTube、GitHub、B站、小红书、V2EX、雪球等；二是突破登录墙与通用搜索盲区，支持需登录的搜索和账号动态查询；三是视频/音频内容提取，包括 B站与 YouTube 字幕、YouTube 视频和小宇宙播客的语音转文字；四是输出干净的结构化数据（如雪球行情、V2EX 帖子回复、GitHub 仓库与 issue）。

适用场景：主要面向使用 CLI 型 AI Agent 的开发者、研究者和内容分析者，用于竞品调研、舆情监控、社区内容聚合、视频内容摘要、中文平台信息采集等需要跨平台、跨形态取数的自动化工作流。

技术亮点：核心思路是“工具编排 + 去 API 化”，把各平台分散的抓取/转写工具封装成统一渠道接口，降低 Agent 的调用复杂度；同时通过预装预测保证开箱可用。项目与多家数据采集服务商（Bright Data、BrowserAct 等）建立赞助合作，侧面反映其在反爬、代理、浏览器自动化等基础设施上的现实依赖。

上手建议：从 README 的“快速开始”按 Python 3.10+ 环境安装，先运行 `agent-reach channels` 确认可用渠道，再挑一个目标平台做最小验证；遇到问题或想新增平台可加入 Discord 社区。贡献方向可优先考虑新平台适配器和现有渠道的稳定性修复。

数据来源：github.com/trending

报告日期：2026-10-26

由 DeepSeek AI 提供智能分析 | 自动生成